LAPORAN PRAKTIKUM
PEMROGRAMAN BERORIENTASI OBJEK
MODUL 3: KELAS DAN OBJEK

Dibuat Oleh :

Nuevalen Refitra Alswando 103072430008

PROGRAM STUDI INFORMATIKA

FAKULTAS INFORMATIKA

UNIVERSITAS TELKOM SURABAYA

2026

3.1 Contoh Kelas Sederhana

Mahasiswa.java

```java
package modul03.mahasiswa;

public class Mahasiswa {  4 usages  new *
    String nama;      // data anggota/atribut  3 usages
    int nim;          // data anggota/atribut  3 usages

    void getNamaMahasiswa() {  2 usages  new *
        System.out.println("Nama: " + nama);
        System.out.println("NIM: " + nim);
    }
}
```

Class Mahasiswa memiliki 2 atribut (nama dan nim) dan 1 method (getNamaMahasiswa) yang berfungsi untuk menampilkan data mahasiswa.

3.2 Main Class untuk Membuat Objek

MainMahasiswa.java

```java
package modul03.mahasiswa;

public class MainMahasiswa {  new *
    public static void main(String[] args) {  new *
        // Membuat objek dari class Mahasiswa
        Mahasiswa mhs1 = new Mahasiswa();

        // Mengisi atribut objek
        mhs1.nama = "Budi Santoso";
        mhs1.nim = 2024001;

        // Memanggil method
        mhs1.getNamaMahasiswa();

        // Membuat objek kedua
        Mahasiswa mhs2 = new Mahasiswa();
        mhs2.nama = "Siti Aminah";
        mhs2.nim = 2024002;
        mhs2.getNamaMahasiswa();
    }
}
```

Output

```
"C:\Program Files\Java\jdk-25\bin\java.exe" "-javaagent:C:\Program Files\JetBrains\IntelliJ IDEA 2025.3.3\lib\idea_rt.jar=56037" -Dfile.encoding=U
Nama: Budi Santoso
NIM: 2024001
Nama: Siti Aminah
NIM: 2024002

Process finished with exit code 0
```

Kode ini mendemonstrasikan cara membuat objek dari class Mahasiswa menggunakan keyword new, mengisi nilai atribut, dan memanggil method untuk menampilkan data.

3.3 Class Field dan Instance Field

Circle.java

```java
package modul03.circle;

public class Circle {  6 usages  new *
    // Class Field (static)
    public static final double PI = 3.14159;  4 usages

    // Instance Field
    public double r;  // jari-jari  7 usages

    // Method untuk menghitung luas
    public double area() {  2 usages  new *
        return PI * r * r;
    }

    // Method untuk menghitung keliling
    public double circumference() {  2 usages  new *
        return 2 * PI * r;
    }

    // Static method
    public static double radiansToDegrees(double rads) {  1 usage  new *
        return rads * 180 / PI;
    }
}
```

MainCircle.java

```java
package modul03.circle;

public class MainCircle {  new *
    public static void main(String[] args) {  new *
        // Membuat objek Circle pertama
        Circle lingkaran1 = new Circle();
        lingkaran1.r = 5.0;

        System.out.println("Lingkaran 1:");
        System.out.println("Jari-jari: " + lingkaran1.r);
        System.out.println("Luas: " + lingkaran1.area());
        System.out.println("Keliling: " + lingkaran1.circumference());

        // Membuat objek Circle kedua
        Circle lingkaran2 = new Circle();
        lingkaran2.r = 10.0;

        System.out.println("\nLingkaran 2:");
        System.out.println("Jari-jari: " + lingkaran2.r);
        System.out.println("Luas: " + lingkaran2.area());
        System.out.println("Keliling: " + lingkaran2.circumference());

        // Mengakses class field dan static method
        System.out.println("\nNilai PI: " + Circle.PI);
        System.out.println("1 radian = " + Circle.radiansToDegrees( rads: 1) + " derajat");
    }
}
```

Output

```
"C:\Program Files\Java\jdk-25\bin\java.exe" "-javaagent:C:\Program Files\JetBrains\IntelliJ IDEA 2025.3.3\lib\idea_rt.jar=59778" -Dfile.encoding=U
Lingkaran 1:
Jari-jari: 5.0
Luas: 78.53975
Keliling: 31.4159

Lingkaran 2:
Jari-jari: 10.0
Luas: 314.159
Keliling: 62.8318

Nilai PI: 3.14159
1 radian = 57.295827908797776 derajat

Process finished with exit code 0
```

Class field PI dideklarasikan sebagai static sehingga terkait dengan class, bukan dengan objek. Instance field r (jari-jari) dimiliki oleh setiap objek Circle sehingga tiap objek memiliki nilainya sendiri. Sementara itu, static method radiansToDegrees dapat dipanggil tanpa perlu membuat objek dari class tersebut.

3.4 Pass by Value dan Pass by Reference

TestPass.java

```java
package modul03.pass;

public class TestPass {  3 usages  new *
    int i, j;  6 usages

    // Constructor
    TestPass(int a, int b) {  1 usage  new *
        i = a;
        j = b;
    }

    // Passed by value dengan parameter tipe data primitif
    void calculate(int m, int n) {  1 usage  new *
        m = m * 10;
        n = n / 2;
        System.out.println("Di dalam method (by value):");
        System.out.println("m = " + m);
        System.out.println("n = " + n);
    }

    // Passed by reference dengan parameter tipe data class
    void calculate(TestPass e) {  1 usage  new *
        e.i = e.i * 10;
        e.j = e.j / 2;
        System.out.println("Di dalam method (by reference):");
        System.out.println("e.i = " + e.i);
        System.out.println("e.j = " + e.j);
    }
}
```

PassedByValue.java

```java
public class PassedByValue {
    public static void main(String[] args) {
        System.out.println("x = " + x);
        System.out.println("y = " + y);

        // Passed by value
        z.calculate(x, y);

        System.out.println("\nNilai sesudah passed by value:");
        System.out.println("x = " + x);
        System.out.println("y = " + y);

        System.out.println("\n=== PASS BY REFERENCE ===");
        System.out.println("Nilai sebelum passed by reference:");
        System.out.println("z.i = " + z.i);
        System.out.println("z.j = " + z.j);

        // Passed by reference
        z.calculate(z);

        System.out.println("\nNilai sesudah passed by reference:");
        System.out.println("z.i = " + z.i);
        System.out.println("z.j = " + z.j);
    }
}
```

Output

```
Run  PassedByValue

"C:\Program Files\Java\jdk-25\bin\java.exe" "-javaagent:C:\Program Files\JetBrains\IntelliJ IDEA 2025.3.3\lib\idea_rt.jar=55971" -Dfile.encod
=== PASS BY VALUE ===
Nilai sebelum passed by value:
x = 10
y = 20
Di dalam method (by value):
m = 100
n = 10

Nilai sesudah passed by value:
x = 10
y = 20

=== PASS BY REFERENCE ===
Nilai sebelum passed by reference:
z.i = 50
z.j = 100
Di dalam method (by reference):
e.i = 500
e.j = 50

Nilai sesudah passed by reference:
z.i = 500
z.j = 50

Process finished with exit code 0
```

Pass by Value berarti method hanya menerima salinan nilai sehingga perubahan di dalam method tidak mempengaruhi variabel asli (terjadi pada tipe data primitif). Sedangkan Pass by Reference berarti yang dilewatkan adalah referensi objek, sehingga perubahan di dalam method akan mempengaruhi objek asli (terjadi pada objek dan array).

3.5 Keyword "this"

Lagu.java

```java
package modul03.keyword;

public class Lagu {  4 usages  new *
    private String pencipta;  3 usages
    private String judul;  3 usages

    // Method dengan parameter yang sama nama dengan atribut
    public void IsiParam(String judul, String pencipta) {  2 usages  new *
        // Keyword 'this' digunakan untuk membedakan atribut dan parameter
        this.judul = judul;
        this.pencipta = pencipta;
    }

    public void cetakKeLayar() {  2 usages  new *
        if (judul == null && pencipta == null) {
            return;
        }
        System.out.println("Judul: " + judul + ", Pencipta: " + pencipta);
    }
}
```

DemoLagu.java

```java
package modul03.keyword;

public class DemoLagu {  new *
    public static void main(String[] args) {  new *
        Lagu a = new Lagu();
        a.IsiParam( judul: "God Will Make A Way",  pencipta: "Don Moen");
        a.cetakKeLayar();

        Lagu b = new Lagu();
        b.IsiParam( judul: "Sempurna",  pencipta: "Andra and The Backbone");
        b.cetakKeLayar();
    }
}
```

Output

```
Run   DemoLagu
"C:\Program Files\Java\jdk-25\bin\java.exe" "-javaagent:C:\Program Files\JetBrains\IntelliJ IDEA 2025.3.3\lib\idea_rt.jar=51699" -Dfile.encod
Judul: God Will Make A Way, Pencipta: Don Moen
Judul: Sempurna, Pencipta: Andra and The Backbone

Process finished with exit code 0
```

Keyword this digunakan untuk mereferensikan objek itu sendiri. Dalam contoh ini, this diperlukan karena nama parameter sama dengan nama atribut class. this.judul merujuk pada atribut class, sedangkan judul merujuk pada parameter method.

3.6 Constructor dengan Keyword "this"

Constructor.java

```java
package modul03.keyword;

public class Constructor {
    private String pengarang;
    private String judul;

    // Constructor tanpa parameter
    private Constructor() {
        // this digunakan untuk memanggil konstruktor lain
        this("Rumah Kita", "GoodBles");
    }

    // Constructor dengan dua parameter
    private Constructor(String judul, String pengarang) {
        this.judul = judul;
        this.pengarang = pengarang;
    }

    private void cetakKeLayar() {
        System.out.println("Judul: " + judul + " Pengarang: " + pengarang);
    }

    public static void main(String[] args) {
        Constructor a, b;

        // Menggunakan constructor dengan parameter
        a = new Constructor("Jurassic Park", "Michael Crichton");

        // Menggunakan constructor tanpa parameter (yang memanggil constructor lain)
        b = new Constructor();

        a.cetakKeLayar();
        b.cetakKeLayar();
    }
}
```

Output

```
Run  Constructor

"C:\Program Files\Java\jdk-25\bin\java.exe" "-javaagent:C:\Program Files\JetBrains\IntelliJ IDEA 2025.3.3\lib\idea_rt.jar=51694" -Dfile.encod
Judul: Jurassic Park Pengarang: Michael Crichton
Judul: Rumah Kita Pengarang: GoodBles

Process finished with exit code 0
```

Dalam contoh ini, keyword this digunakan untuk memanggil constructor lain dalam class yang sama (this("Rumah Kita", "GoodBles")) serta untuk membedakan antara atribut class dan parameter method, seperti pada this.judul = judul.

3.7 Array Bertipe Objek

Manusia.java

```java
package modul03.array;

public class Manusia {  3 usages  new *
    private String nama;  2 usages
    private int tinggi;  2 usages

    void setInfo(String nama, int tinggi) {  4 usages  new *
        this.nama = nama;
        this.tinggi = tinggi;
    }

    void info() {  5 usages  new *
        System.out.println(this.nama + " Memiliki Tinggi " + this.tinggi + "cm");
    }
}
```

MainManusia.java

```java
package modul03.array;

public class MainManusia {  new *
    public static void main(String[] args) {  new *
        // Membuat array dari objek Manusia
        Manusia[] manusia = new Manusia[4];

        // Inisialisasi setiap elemen array
        for (int i = 0; i < manusia.length; i++) {
            manusia[i] = new Manusia();
        }

        // Mengisi data setiap objek
        manusia[0].setInfo( nama: "Hermawan",  tinggi: 180);
        manusia[1].setInfo( nama: "Suciati",  tinggi: 160);
        manusia[2].setInfo( nama: "Boy",  tinggi: 170);
        manusia[3].setInfo( nama: "Neneng",  tinggi: 165);

        // Menampilkan informasi setiap objek
        System.out.println("=== DATA MANUSIA ===");
        manusia[0].info();
        manusia[1].info();
        manusia[2].info();
        manusia[3].info();

        // Atau menggunakan loop
        System.out.println("\n=== MENGGUNAKAN LOOP ===");
        for (int i = 0; i < manusia.length; i++) {
            manusia[i].info();
        }
    }
}
```

Output

```
Run   MainManusia
"C:\Program Files\Java\jdk-25\bin\java.exe" "-javaagent:C:\Program Files\JetBrains\IntelliJ IDEA 2025.3.3\lib\idea_rt.jar=53105" -Dfile.encod
=== DATA MANUSIA ===
Hermawan Memiliki Tinggi 180cm
Suciati Memiliki Tinggi 160cm
Boy Memiliki Tinggi 170cm
Neneng Memiliki Tinggi 165cm

=== MENGGUNAKAN LOOP ===
Hermawan Memiliki Tinggi 180cm
Suciati Memiliki Tinggi 160cm
Boy Memiliki Tinggi 170cm
Neneng Memiliki Tinggi 165cm

Process finished with exit code 0
```

Array dapat menyimpan beberapa objek dari kelas yang sama. Setiap elemen array harus diinisialisasi dengan new Manusia() sebelum digunakan. Ini menunjukkan konsep array bertipe objek.

3.8 Enkapsulasi dengan Access Modifier

Student.java

```java
public class Student {
    private String nim;
    private String nama;
    private final Integer MAX_BALANCE = 10000;

    // Constructor
    Student(String paramNim, String paramNama) {
        nim = paramNim;
        nama = paramNama;
    }

    // Public methods untuk mengakses private data
    void sayHello() {
        System.out.println("Hello");
    }

    void sayGoodbye(String paramNama) {
        System.out.println("Goodbye " + paramNama + " remember to study!");
    }

    String getNama() {
        return nama;
    }

    String getNim() {
        return nim;
    }

    void setNama(String nama) {
        this.nama = nama;
    }

    void setNim(String nim) {
        this.nim = nim;
    }

    Integer getMaxBalance() {
        return MAX_BALANCE;
    }
}
```

MainApp.java

```java
package modul03.enkapsulasi;

public class MainApp {
    public static void main(String[] args) {
        // Membuat objek Student
        var student1 = new Student("2301", "Dani");
        Student student2 = new Student("2302", "Rina");

        // Mengakses data melalui getter
        System.out.println("=== DATA STUDENT 1 ===");
        System.out.println("NIM: " + student1.getNim());
        System.out.println("Nama: " + student1.getNama());
        System.out.println("Max Balance: " + student1.getMaxBalance());

        System.out.println("\n=== DATA STUDENT 2 ===");
        System.out.println("NIM: " + student2.getNim());
        System.out.println("Nama: " + student2.getNama());

        // Memanggil methods
        System.out.println("\n=== MEMANGGIL METHODS ===");
        student1.sayHello();
        student1.sayGoodbye("Joko");
        student2.sayHello();

        // Mengubah data melalui setter
        System.out.println("\n=== SETTING ULANG DATA ===");
        student1.setNama("Dani Updated");
        student1.setNim("2301A");
        System.out.println("NIM Baru: " + student1.getNim());
        System.out.println("Nama Baru: " + student1.getNama());
    }
}
```

Private modifier digunakan untuk melindungi data agar tidak dapat diakses langsung dari luar class (enkapsulasi). Public method seperti getter dan setter digunakan untuk mengakses serta mengubah data yang bersifat private. Sementara itu, final modifier pada MAX_BALANCE menunjukkan bahwa nilai tersebut merupakan konstanta yang tidak dapat diubah.